

Mini Project Review I

Sem-V

EcoShield: A Risk-Adaptive Network Security System for Energy-Efficient Threat Monitoring

Group Members

Nishaad Desai	124AX013
Anish Chaudhary	124AX010
Ajay Nadar	225AX067
Arjun Dongre	124AX018

Guide: Prof. Senthil

Introduction

- Network security tools such as IDS/IPS are traditionally run at a fixed, always-on level of inspection intensity, regardless of how risky the current traffic actually is.
- This constant high-intensity monitoring consumes significant CPU, memory, processing time and energy – even during long periods of normal, low-risk activity.
- EcoShield is a web-based, risk-adaptive network security system that changes the intensity of monitoring and security controls according to the current threat level.
- It continuously observes traffic, computes a risk score, and switches between three security modes – Baseline, Elevated and Intensive – instead of always running at maximum intensity.
- Alongside security state, EcoShield also measures and displays the computational and estimated energy/CO₂ cost of its own operation on a live dashboard.
- Goal: reduce unnecessary computational and energy overhead compared to always-on high-intensity security, without compromising detection and response performance.

Paper No	Year	Approach Used	Key Findings	Limitations
1	2012	Y. Ma, W. Jiang, N. Sang , “An Adaptive Risk Control and Security Management for Embedded Real-Time System,” <i>7th Int’l Conf. on Availability, Reliability and Security (ARES)</i> , Prague, 2012, pp. 11–17. A two-level feedback scheduling framework dynamically assigns and adjusts the security level of runtime tasks based on system risk.	Adaptive risk-based scheduling reduces system risk while preserving soft real-time performance, showing good adaptivity under dynamic workloads.	Targets uniprocessor task scheduling in embedded systems, not network traffic; does not measure network-level CPU/energy overhead of security operations.
2	2016	E. Viegas, A. Santin, A. França, R. Jasinski, V. Pedroni, L. Oliveira , “Towards an Energy-Efficient Anomaly-Based Intrusion Detection Engine for Embedded Systems,” <i>IEEE Transactions on Computers</i> , vol. 66, no. 1, pp. 163–177, 2016. Hardware implementation of feature extraction and ML classifiers (DT, NB, kNN) for anomaly-based IDS, with detailed SW-vs-HW energy measurements.	Hardware implementation cuts energy consumption significantly versus an equivalent software version; the new feature extractor uses only ~22% of the energy of prior methods.	Requires custom FPGA/hardware implementation rather than a software-only, dynamically adaptive intensity approach; detection depth is not adjusted in real time to network risk.
3	2012	E. Albin, N. C. Rowe , “A Realistic Experimental Comparison of the Suricata and Snort Intrusion-Detection Systems,” <i>26th Int’l Conf. on Advanced Information Networking and Applications Workshops (WAINA)</i> , 2012, pp. 122–127. Real-world deployment comparison of Suricata vs. Snort measuring speed, memory and accuracy under live traffic and red-team attacks.	Suricata handles larger traffic volumes than Snort with similar accuracy, and its performance scales roughly linearly with processor cores up to 48.	Both IDS run at a fixed configuration throughout testing; the study does not explore adapting rule-set or inspection depth to network risk to save resources.
4	2020	D. Fadhilah, M. I. Marzuki , “Performance Analysis of IDS Snort and IDS Suricata with Many-Core Processor in Virtual Machines Against DoS/DDoS Attacks,” <i>2nd Int’l Conf. on Broadband Communications, Wireless Sensors and Powering (BCWSP)</i> , 2020, pp. 157–162. Tests Snort v3 and Suricata under 1/2/4-core VM configurations, measuring CPU/memory use and detection under DoS/DDoS traffic.	CPU and memory consumption of both IDS engines scale with core count and traffic load, confirming that available compute resources materially shift the detection/resource trade-off.	Core allocation is fixed per experiment rather than dynamically adjusted based on live risk; no persistence/hysteresis mechanism for switching detection intensity.

Problem Statement

Conventional network security monitoring may maintain similar monitoring intensity even when network risk changes.

High-intensity inspection, IDS processing and extensive logging can introduce unnecessary computational overhead during low-risk network activity.

EcoShield proposes dynamically adjusting security intensity according to observed network risk while maintaining a minimum security baseline.

OBJECTIVES

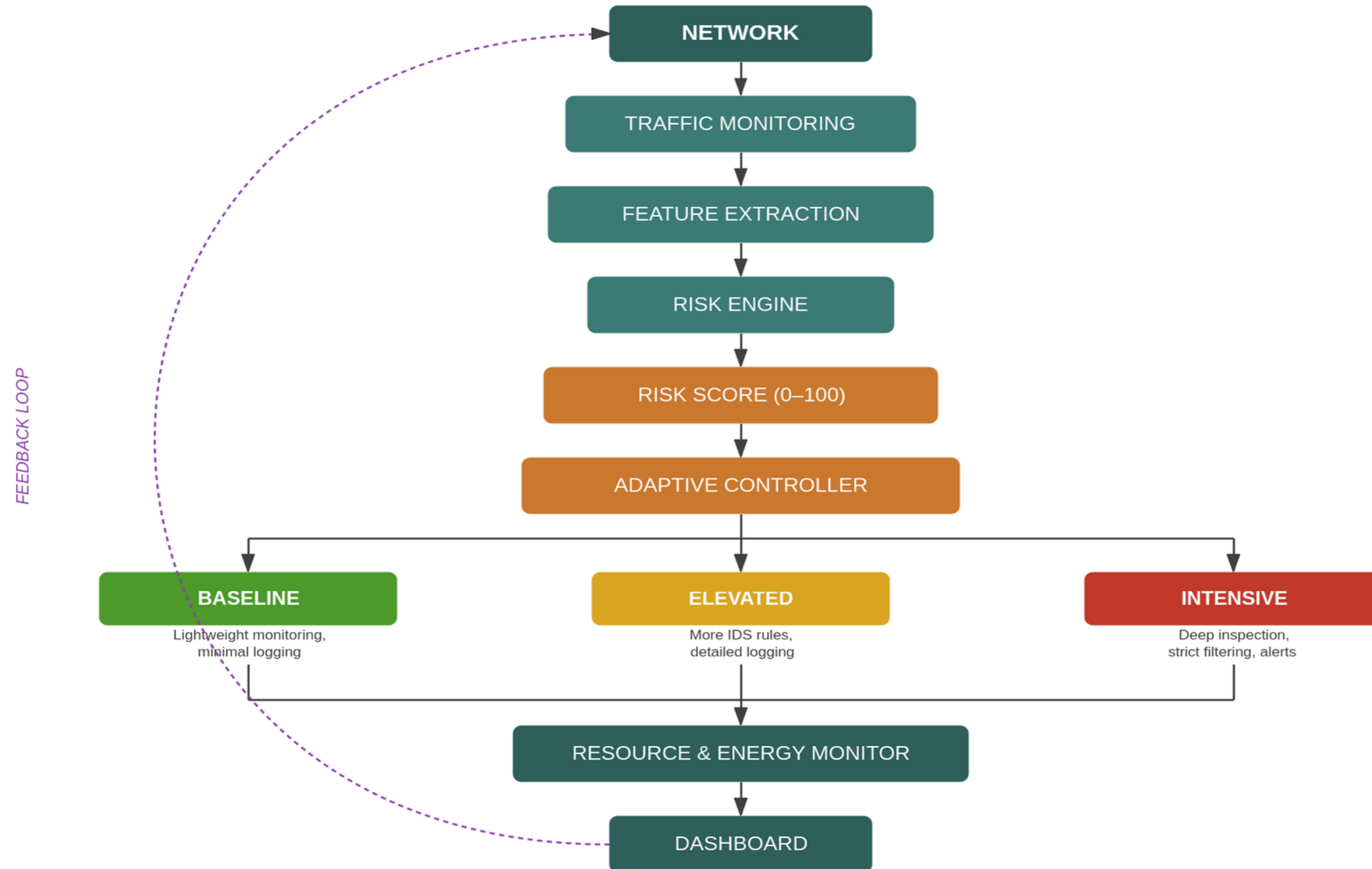
- To continuously monitor network traffic and IDS observations.
- To extract relevant traffic and threat indicators for risk assessment.
- To calculate a continuously updated network risk score on a 0–100 scale.
- To dynamically select Baseline, Elevated, or Intensive security modes based on network risk.
- To maintain a minimum level of security while reducing unnecessary monitoring overhead
- To measure CPU, memory, processing time, and estimated energy/CO₂ overhead.
- To provide a web dashboard for monitoring risk, security state, alerts, and resource usage.

Proposed System

Use these EcoShield points:

- Monitors network traffic and IDS observations.
- Extracts traffic and threat indicators.
- Calculates a continuously updated network risk score.
- Dynamically selects Baseline, Elevated or Intensive security modes.
- Measures CPU, memory, processing time and estimated energy overhead.
- Displays risk, security state, alerts and resource usage through a web dashboard.

FLOWCHART



Methodology

1. Traffic Monitoring

Network traffic and IDS observations (packet/flow rates, protocols, connection behaviour, alerts) are continuously captured.

2. Feature Extraction

Traffic and threat indicators are derived from raw observations to feed the risk engine.

3. Risk Scoring

A transparent weighted risk score (0–100) is calculated from network features, anomaly indicators and IDS alerts.

4. Mode Selection

Thresholds with hysteresis/persistence select Baseline, Elevated or Intensive mode, avoiding rapid oscillation between modes.

5. Control Application

Baseline → lightweight monitoring. Elevated → additional logging and IDS rules. Intensive → deep packet inspection, strict filtering and alert escalation.

6. Resource & Energy Measurement

CPU, memory, processing time and estimated energy/CO₂ are measured for the currently active mode.

7. Feedback

Results are logged and displayed on the dashboard; the cycle repeats continuously, letting EcoShield reassess and re-adapt in real time.

Software

Network/Traffic

- Scapy
- Mininet
- Docker
- Wireshark

Detection Layer

- Suricata
- Zeek

Risk Engine

- Python
- Scikit-learn

Backend

- FastAPI

Frontend

- React
- Recharts / Chart.js

Database

- SQLite / PostgreSQL

Resource Monitoring

- psutil
- Linux system metrics

Conclusion

- EcoShield dynamically adapts security intensity based on changing network risk and threat conditions.
- The system evaluates detection performance alongside CPU, memory, processing time, and estimated energy consumption.
- EcoShield aims to balance effective cybersecurity with reduced computational overhead through risk-aware security control.

Future Scope

- ML Risk Scoring: Use machine learning for smarter risk assessment.
- Direct Energy Measurement: Use sensors for accurate power measurement.
- Automated Response: Add automatic blocking and device isolation.
- Distributed Deployment: Extend EcoShield across multiple network segments.
- Threat Prediction: Predict attacks using historical network patterns.
- Carbon-Aware Security: Optimize security based on energy and carbon intensity.

References

- [1] Y. Ma, W. Jiang, and N. Sang, “An Adaptive Risk Control and Security Management for Embedded Real-Time System,” in *Proc. 7th Int’l Conf. on Availability, Reliability and Security (ARES)*, Prague, Czech Republic, 2012, pp. 11–17.
- [2] E. Viegas, A. Santin, A. França, R. Jasinski, V. Pedroni, and L. Oliveira, “Towards an Energy-Efficient Anomaly-Based Intrusion Detection Engine for Embedded Systems,” *IEEE Transactions on Computers*, vol. 66, no. 1, pp. 163–177, 2016.
- [3] E. Albin and N. C. Rowe, “A Realistic Experimental Comparison of the Suricata and Snort Intrusion-Detection Systems,” in *Proc. 26th Int’l Conf. on Advanced Information Networking and Applications Workshops (WAINA)*, 2012, pp. 122–127.
- [4] D. Fadhilah and M. I. Marzuki, “Performance Analysis of IDS Snort and IDS Suricata with Many-Core Processor in Virtual Machines Against DoS/DDoS Attacks,” in *Proc. 2nd Int’l Conf. on Broadband Communications, Wireless Sensors and Powering (BCWSP)*, 2020, pp. 157–162.
- [5] Open Information Security Foundation, *Suricata User Guide*. [Online]. Available: <https://suricata.io/documentation/>
- [6] G. Rodola, *psutil Documentation*. [Online]. Available: <https://psutil.readthedocs.io/>

THANK YOU